

```

8     min_dist = float( int )
9     pair = None
10
11     n = len(points)
12
13     for i in range(n):
14         for j in range(i + 1, n):
15             d = distance(points[i], points[j])
16
17             if d < min_dist:
18                 min_dist = d
19                 pair = (points[i], points[j])
20
21     return pair, min_dist
22
23
24 n = int(input())
25
26 points = []
27
28 for i in range(n):
29     x, y = map(int, input().split())
30     points.append((x, y))
31
32 pair, dist = closest_pair(points)
33
34 print("Closest Pair:", pair[0], pair[1])
35 print("Minimum Distance:", dist)

```



```

0 0
2 2
Closest Pair: (0, 0) (2, 2)
Minimum Distance: 2.8284271247461903

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

6
7 def closest_pair(points):
8     min_dist = float('inf')
9     pair = None
10
11     for i in range(len(points)):
12         for j in range(i + 1, len(points)):
13             d = distance(points[i], points[j])
14
15             if d < min_dist:
16                 min_dist = d
17                 pair = (points[i], points[j])
18
19     return pair, min_dist
20
21
22 n = int(input())
23
24 points = []
25
26 for i in range(n):
27     x, y = map(int, input().split())
28     points.append((x, y))
29
30 pair, dist = closest_pair(points)
31
32 print("Closest Pair:", pair[0], pair[1])
33 print("Minimum Distance:", dist)

```



```

9 6
1 5
Closest Pair: (7, 8) (9, 6)
Minimum Distance: 2.8284271247461903

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

1 def median(arr):
2     arr.sort()
3     return arr[len(arr) // 2]
4
5
6 def median_of_medians(arr):
7     if len(arr) <= 5:
8         return median(arr)
9
10    groups = []
11
12    for i in range(0, len(arr), 5):
13        group = arr[i:i + 5]
14        groups.append(median(group))
15
16    return median_of_medians(groups)
17
18
19 n = int(input())
20 arr = list(map(int, input().split()))
21
22 result = median_of_medians(arr)
23
24 print("Median =", result)

```

input

```

8
1 2 3 56 68 98 47 5
Median = 47

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

32
33
34 def kth_smallest(arr, k):
35     if len(arr) == 1:
36         return arr[0]
37
38     pivot = median_of_medians(arr)
39
40     smaller, equal, greater = partition(arr, pivot)
41
42     if k <= len(smaller):
43         return kth_smallest(smaller, k)
44
45     elif k <= len(smaller) + len(equal):
46         return pivot
47
48     else:
49         return kth_smallest(
50             greater,
51             k - len(smaller) - len(equal)
52         )
53
54
55 n = int(input())
56 arr = list(map(int, input().split()))
57 k = int(input())
58
59 print("K-th Smallest Element =", kth_smallest(arr, k))

```

input

```

6
1 5 7 8 9 4
5
K-th Smallest Element = 8

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

```

9 def median_of_medians(arr):
10     if len(arr) <= 5:
11         return median(arr)
12
13     medians = []
14
15     for i in range(0, len(arr), 5):
16         medians.append(median(arr[i:i + 5]))
17
18     return median_of_medians(medians)
19
20
21 n = int(input())
22 arr = list(map(int, input().split()))
23
24 start = time.time()
25 m1 = median_of_medians(arr.copy())
26 time1 = time.time() - start
27
28 start = time.time()
29 temp = sorted(arr)
30 m2 = temp[n // 2]
31 time2 = time.time() - start
32
33 print("Median of Medians =", m1)
34 print("Sorting Median =", m2)
35 print("Median of Medians Time =", time1)
36 print("Sorting Time =", time2)

```

input

```

Median of Medians = 26
Sorting Median = 16
Median of Medians Time = 3.600120544433594e-05
Sorting Time = 7.3909759521484375e-06

```